Maps & Hash Functions

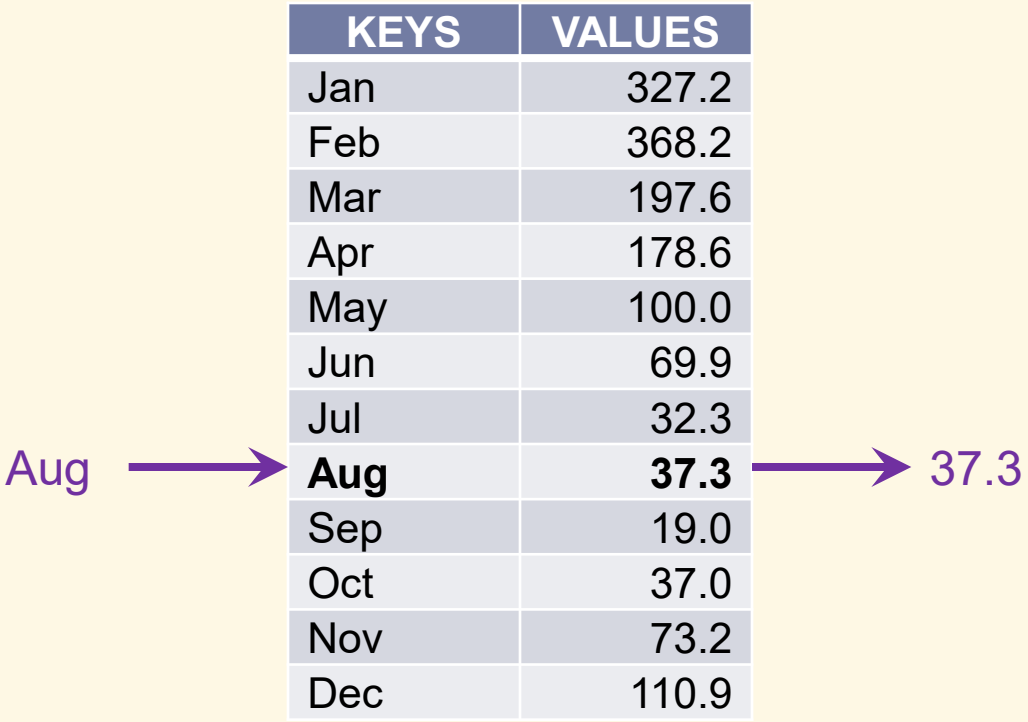
- Map (dictionary) data structure
- Hash functions
- Probing

Overview

- Arrays are integer indexed

➔ How to generalize with different index types?

➔ Pairs of <key, $value$> where key and $value$ can be of any types



Terminology

- These terms may share the same similarities in different programming languages or libraries:
  - Map (C++, Java, Matlab)
  - Dictionary (C#, Python)
  - Associative array (JavaScript, PHP)
  - Lookup table (C#, Lua)
  - Hash table (Ruby)

- Maps are generally single valued (one key → one value)
  - Multi-valued maps can be considered as exercise

String-key dictionary

Introduction

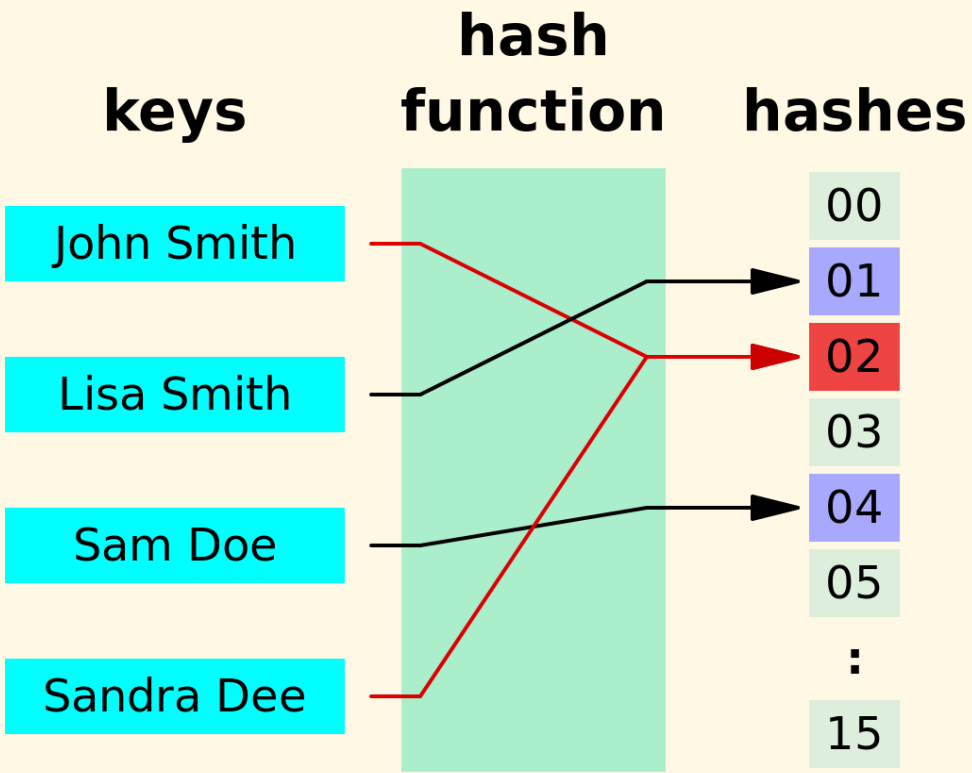
- A special case but very frequently used map data structure
- Mapping from **string keys** to other value types

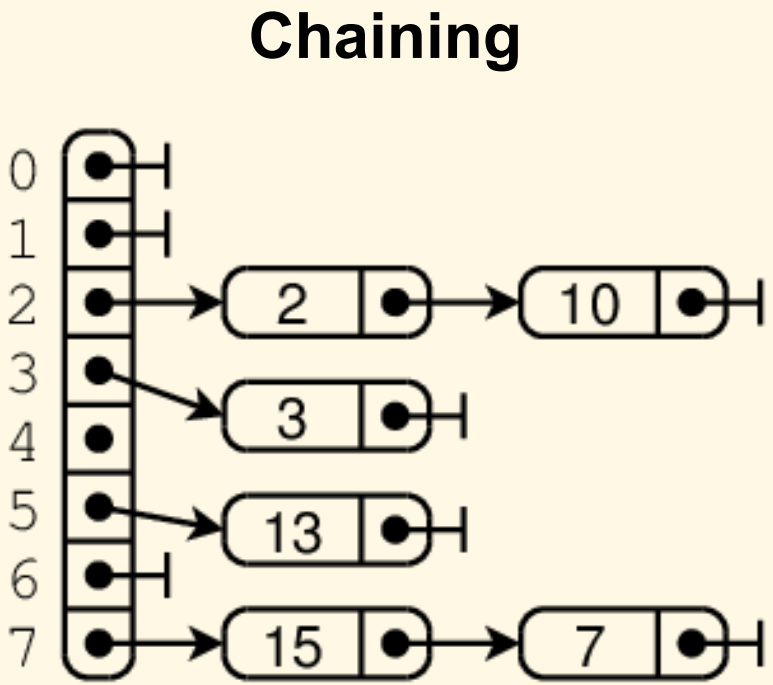
- Main operations:
  - Insert elements
  - Remove elements
  - Find elements with their keys

- Applications:
  - Address book
  - Dictionary
  - Data lookup

Overview

- Arrange the data into groups with deterministic keys





Declarations

```
typedef int (*STRING_HASH_FUNC)(char* key, int size);

typedef struct {
    LLIST* entries;
    int size;
    STRING_HASH_FUNC hasher;
} STR_MAP;

typedef struct {
    char* key;
    void* value;
} STR_MAP_ITEM;
```

Operations

```
void mapInit(STR_MAP* m, int size, STRING_HASH_FUNC hasher);
void mapDestroy(STR_MAP* m);

void mapInsert(STR_MAP* m, char* key, void* value, int sz);
void* mapFind(STR_MAP* m, char* key);

typedef void (*MAP_CALLBACK_FUNC)
         (STR_MAP_ITEM* item, void* user);
void mapForEach(STR_MAP* m,
         MAP_CALLBACK_FUNC callback, void* user);

void mapDelete(STR_MAP* m, char* key);
void mapDeleteAll(STR_MAP* m);
```

Create map

```
void mapInit(STR_MAP* m, int size,
             STRING_HASH_FUNC hasher)
{
    m->size = size;
    m->hasher = hasher == NULL ? defaultHasher : hasher;
    m->entries = (void*)malloc(sizeof(LLIST) * m->size);

    for (int i = 0; i < m->size; i++)
        llInit(&m->entries[i]);
}
```

Destroy

```
void mapDestroy(STR_MAP* m)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], freeListElementData, NULL);
        llDestroy(&m->entries[i]);
    }

    free(m->entries);
}
```

Insert element

```
void mapInsert(STR_MAP* m, char* key, void* value, int sz) {
    LLIST* l = findEntry(m, key);
    LIST_ELEM* e = findElement(m, *l, key);

    if (e == NULL) {
        STR_MAP_ITEM item;
        int keylen = strlen(key);
        item.key = malloc(keylen+1);
        strcpy(item.key, key);

        item.value = malloc(sz);
        memcpy(item.value, value, sz);

        llInsertTail(l, &item, sizeof(item));
    } else {
        STR_MAP_ITEM* item = (STR_MAP_ITEM*)(e->data);
        free(item->value);
        item->value = malloc(sz);
        memcpy(item->value, value, sz);
    }
}
```

Find element

```
void* mapFind(STR_MAP* m, char* key)
{
    LLIST *l = findEntry(m, key);
    LIST_ELEM* e = findElement(m, *l, key);
    return e == NULL ? NULL :
          ((STR_MAP_ITEM*)(e->data))->value;
}
```

Delete element

```
void mapDelete(STR_MAP* m, char* key)
{
  LLIST *l = findEntry(m, key);
  if (*l == NULL) return;

  STR_MAP_ITEM* item =
      (STR_MAP_ITEM*)((*l)->data);
  if (strcmp(item->key, key) == 0) {
    free(item->key);
    free(item->value);
    llDeleteHead(l);
    return;
  }
  LIST_ELEM* e;
  for (e = *l; e->next != NULL;
       e = e->next)
  {
    item = (STR_MAP_ITEM*)
           (e->next->data);
    if (strcmp(item->key, key)==0)
    {
      free(item->key);
      free(item->value);
      llDeleteAfter(l, e);
      break;
    }
  }
}
```

Iteration

```
void mapForEach(STR_MAP* m,
       MAP_CALLBACK_FUNC callback, void* user)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], (LLCALLBACK)callback, NULL);
    }
}
```

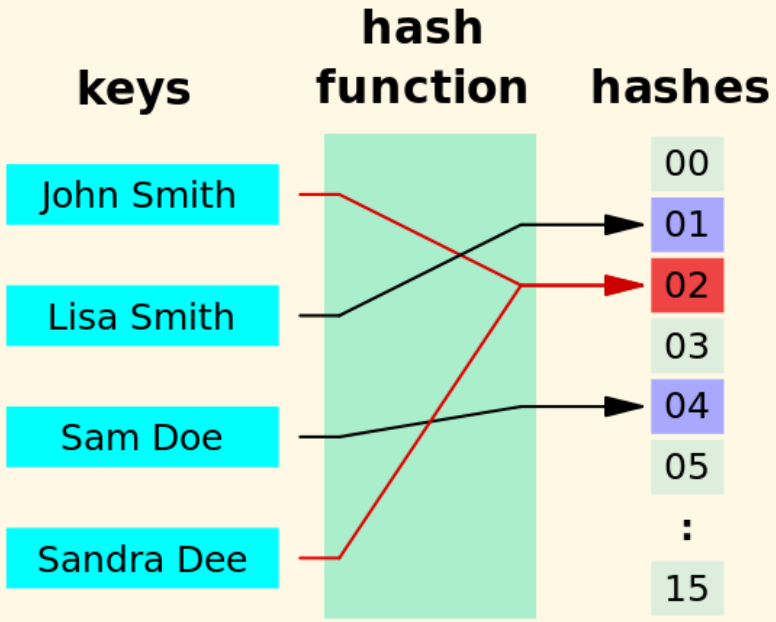
Delete all

```
void mapDeleteAll(STR_MAP* m)
{
    for (int i = 0; i < m->size; i++) {
        llForEach(m->entries[i], freeListElementData, NULL);
        llDeleteAll(&m->entries[i]);
    }
}
```

Hash functions

Definition

- Map from values of any type to fixed-size integers



- Usually composed of two functions
  - Hash code mapping: keys → integers
  - Compression mapping: integers → $[0..N-1]$

Good hash functions?

- Properties
  - Uniformity (probability distribution)
  - Efficiency
- Frequently used functions
  - Polynomial
  - Permutation
  - Logical operations (e.g., XOR)
- Attn: Variable-length keys can be resampled to become fixed-length before hashing

Generalized maps

Generalized key types

- Any type with following two operations can be used as key:
  - Hashing
  - Comparison for equality

- Examples:
  - Floating point values
  - 2D, 3D points
  - Structures
  - Binary data

Other map types

- Multi-valued maps
  - Aka multimap, multihash, multidict

- Two-way maps
  - Roles of keys and values are interchangeable

Exercises

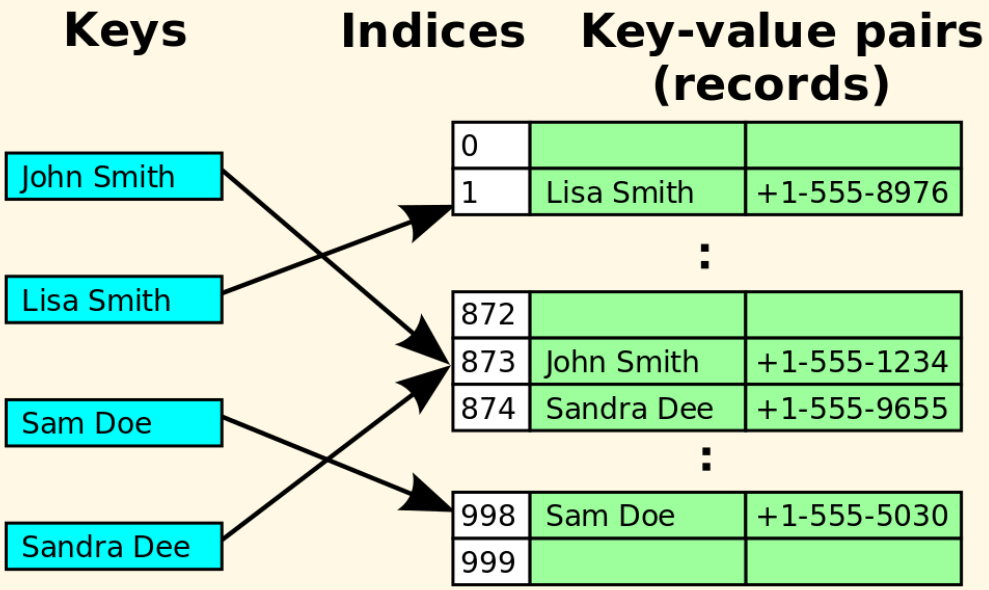
- Implement a generic hash table in C.
- Implement a generic hash table in C++.
- Implement a hash table where the keys are 3D points with `float`-typed components.

Probing

Introduction

- Linked-list based maps may be too complicated for implementation on resource-limited platforms or hardware

➔ Array based hash tables

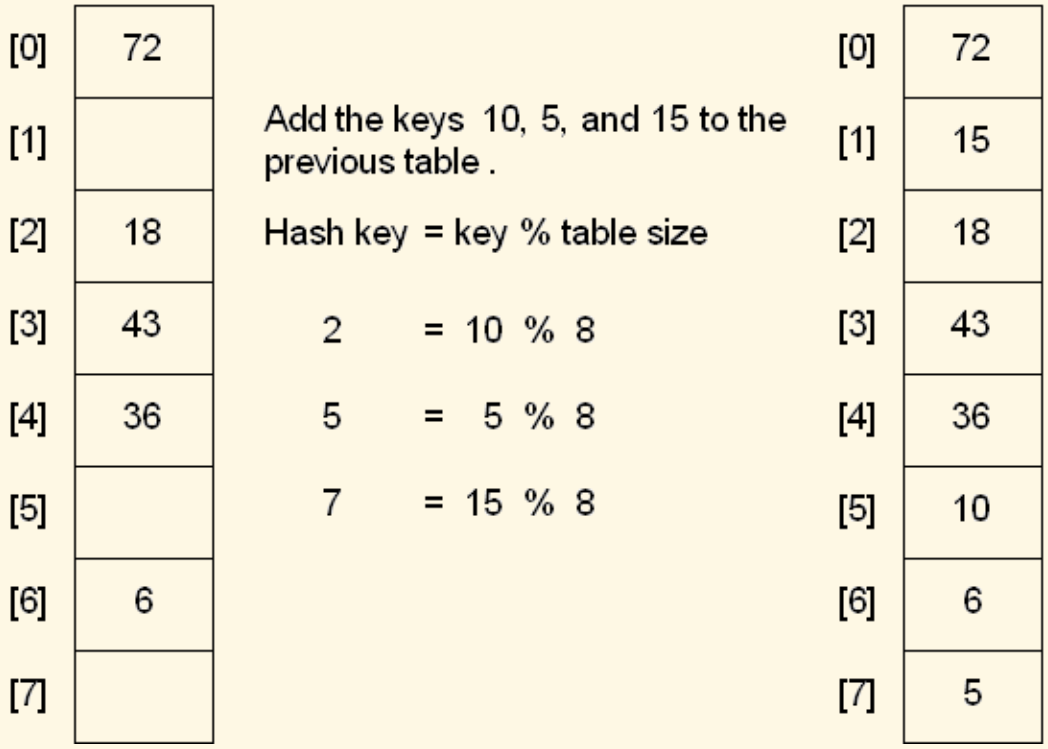


- Need to resolve hash collisions?

➔ Probing methods

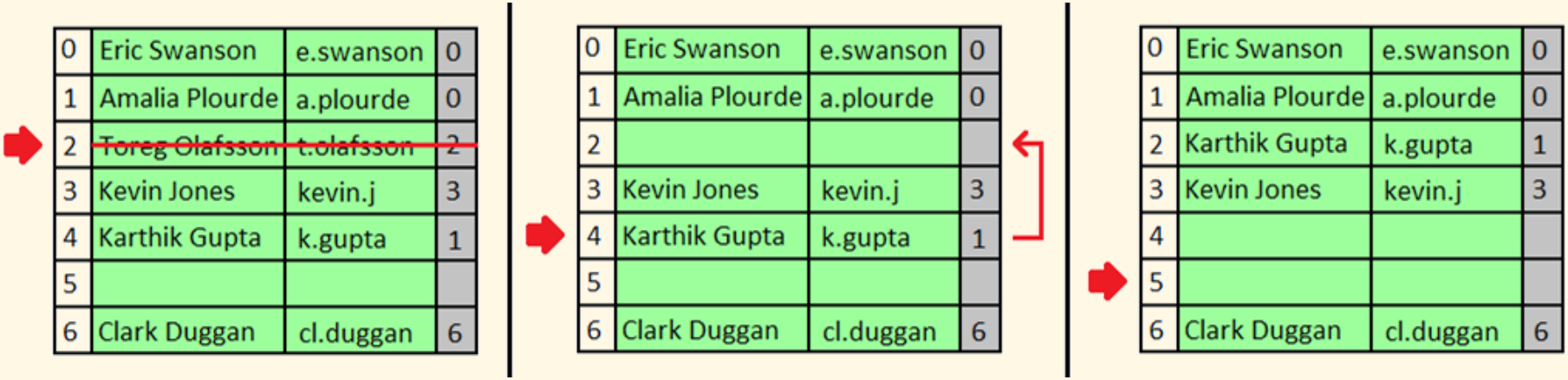
Linear probing: Insertion & Searching

- When insertion causes a collision, search the table for the closest following free location



Linear probing: Deletion

- Deletion may cause problems to searching
- Solution 1:
  - Search forward for an element whose hash is ≤ that of the deleted element, and move it backward for replace
  - Do the same to the moved element (if any)…



0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2	~~Toreg Olafsson~~	~~t.olafsson~~	~~2~~
3	Kevin Jones	kevin.j	3
4	Karthik Gupta	k.gupta	1
5			
6	Clark Duggan	cl.duggan	6

0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2			
3	Kevin Jones	kevin.j	3
4	Karthik Gupta	k.gupta	1
5			
6	Clark Duggan	cl.duggan	6

0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2	Karthik Gupta	k.gupta	1
3	Kevin Jones	kevin.j	3
4			
5			
6	Clark Duggan	cl.duggan	6

- Solution 2: Lazy deletion with the help of a flag

Quadratic probing

- Linear probing tends to have clustering problem

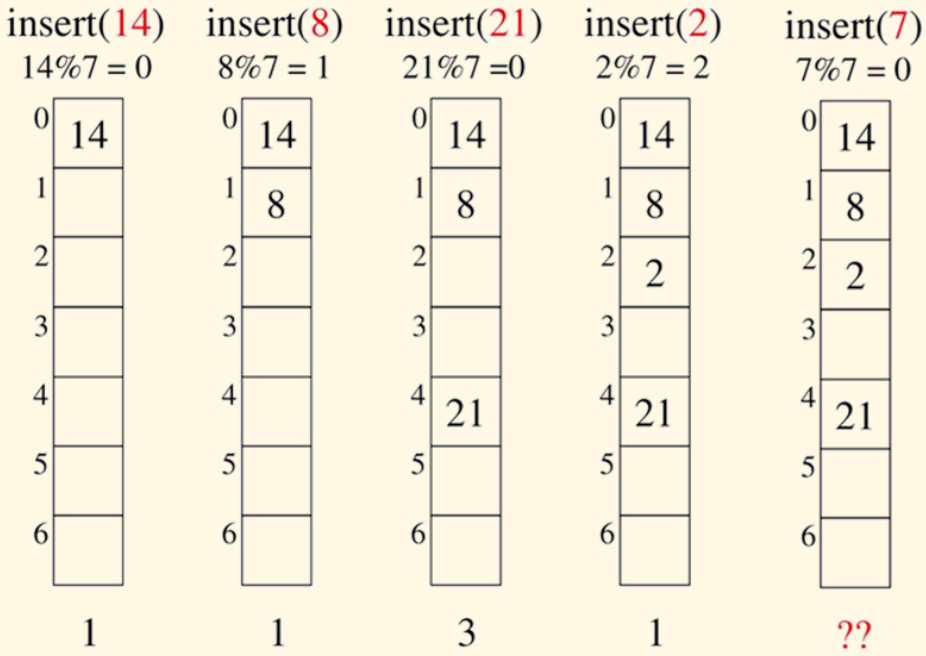
➔ Use a quadratic function to calculate the i^{th} probe position for a key k:

$$p(k, i) = (h(k) + i^2) \bmod N$$

where

- N: array size, better to be prime number
- $h(k)$: hash function for key k

- Not guaranteed to succeed when map is half full
- Must use lazy deletion



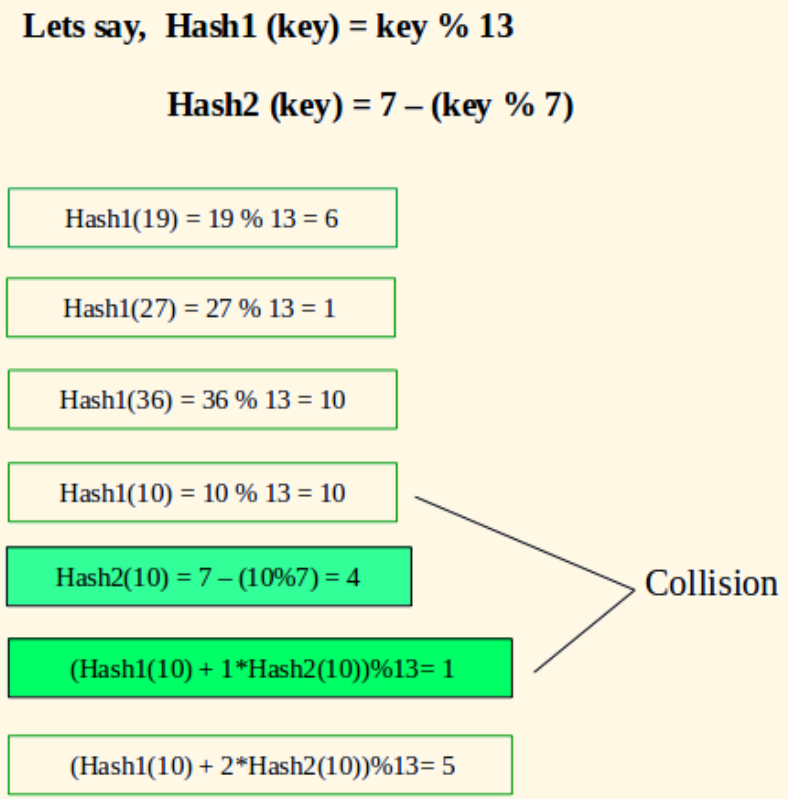
Double hashing

- Use a second hash function to resolve hash collisions:

$$p(k,i) = \left(h_1(k) + i \times h_2\,(k)\right) \bmod N$$

where

- $h_2(k)$ should never return 0



Homework

- Implement multiple hash tables for a student list so that we can find an item using different properties.